Experiment No. 10
Exploring and Implementing Large Language Model

Intelligent Document-Based AI Chatbot using LLM and RAG

```
!pip install -q langchain langchain-groq langgraph
import os
from getpass import getpass
# Option 1: Colab Secrets (key icon on left sidebar -> add GROQ_API_KEY)
# Option 2: type the key when asked
try:
    from google.colab import userdata
    os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
    print("API key loaded from Colab Secrets")
except Exception:
    os.environ["GROQ_API_KEY"] = getpass("Enter Groq API key: ")
```

Code — Part 2: Initialize LLM

```
from groq import Groq
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Fetch the models currently available on your Groq account
available = [m.id for m in Groq().models.list().data]
skip = ["whisper", "tts", "guard", "playai", "orpheus", "compound"]
chat_models = [m for m in available if not any(x in m for x in skip)]

print("Available chat models:", chat_models)

# Pick the first preferred model that exists, else any chat model
preferred = [
    "llama-3.3-70b-versatile",
    "openai/gpt-oss-20b",
    "openai/gpt-oss-120b",
    "meta-llama/llama-4-scout-17b-16e-instruct",
    "qwen/qwen3-32b"
]

MODEL = next((m for m in preferred if m in chat_models), chat_models[0])
print("Using model:", MODEL)

llm = ChatGroq(model=MODEL, temperature=0.7)

response = llm.invoke(
    "What is a Large Language Model? Answer in 2 lines."
)

print(response.content)
```

Code — Part 3: LangChain Prompt Chain

```
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} to a beginner in 3 points."
)

chain = prompt | llm | StrOutputParser()
```

```
print(chain.invoke({
    "topic": "Transformer architecture"
}))
```

Code — Part 4: Sequential Chain

```
title_prompt = ChatPromptTemplate.from_template(
    "Give one catchy blog title about {topic}. Only give the title."
)

intro_prompt = ChatPromptTemplate.from_template(
    "Write a 4-line introduction for a blog titled: {title}"
)

title_chain = title_prompt | llm | StrOutputParser()

seq_chain = (
    {"title": title_chain}
    | intro_prompt
    | llm
    | StrOutputParser()
)

topic = "AI in education"

print("Title:", title_chain.invoke({"topic": topic}))

print(
    "\nIntroduction:\n",
    seq_chain.invoke({"topic": topic})
)
```

Code — Part 5: LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# State: shared data between nodes
class State(TypedDict):
    question: str
    category: str
    answer: str


# Node 1: classify the question
def classify(state: State):
    res = llm.invoke(
        "Classify the question as 'tech' or 'general'. "
        "Reply with one word only.\n"
        f"Question: {state['question']}"
    ).content.lower()

    return {
        "category": "tech" if "tech" in res else "general"
    }


# Node 2: technical answer
```

```
def tech_node(state: State):
    ans = llm.invoke(
        f"You are a technical expert. "
        f"Answer briefly: {state['question']}"
    )

    return {
        "answer": ans.content
    }


# Node 3: general answer
def general_node(state: State):
    ans = llm.invoke(
        f"You are a friendly assistant. "
        f"Answer briefly: {state['question']}"
    )

    return {
        "answer": ans.content
    }


# Router for conditional edge
def route(state: State):
    return state["category"]


# Build the graph
graph = StateGraph(State)

graph.add_node("classify", classify)
graph.add_node("tech", tech_node)
graph.add_node("general", general_node)

graph.add_edge(START, "classify")

graph.add_conditional_edges(
    "classify",
    route,
    {
        "tech": "tech",
        "general": "general"
    }
)

graph.add_edge("tech", END)
graph.add_edge("general", END)

app = graph.compile()

print("Graph compiled successfully")
```

Code — Part 6: Test LangGraph

```
questions = [
    "What is self-attention in transformers?",
```

```
    "Suggest a good weekend hobby"
]

for q in questions:
    result = app.invoke({
        "question": q
    })

    print("Q:", q)
    print("Category:", result["category"])
    print("A:", result["answer"])
    print("-" * 60)
```

Code — Part 7: RAG Setup

```
!pip install -q gradio pypdf langchain-text-splitters langchain-huggingface sentence-transformers
from pypdf import PdfReader
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings

# 1. Embedding model
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# 2. Text splitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150
)

# Knowledge base
KB = {
    "store": None,
    "name": None,
    "summary": "",
    "pages": 0,
    "chunks": 0
}


def load_file(path):
    """Read a PDF or TXT file and return one Document per page."""

    if path.lower().endswith(".pdf"):
        reader = PdfReader(path)

        docs = [
            Document(
                page_content=p.extract_text() or "",
                metadata={"page": i + 1}
            )
            for i, p in enumerate(reader.pages)
        ]
```

```
    else:
        with open(
            path,
            encoding="utf-8",
            errors="ignore"
        ) as f:
            docs = [
                Document(
                    page_content=f.read(),
                    metadata={"page": 1}
                )
            ]

    return [
        d for d in docs
        if d.page_content.strip()
    ]


def build_knowledge_base(docs, name):
    """Chunk -> Embed -> Store in vector store."""

    if not docs:
        raise ValueError(
            "No readable text found (scanned PDFs are not supported)."
        )

    chunks = splitter.split_documents(docs)

    store = InMemoryVectorStore(embeddings)

    store.add_documents(chunks)

    full_text = " ".join(
        d.page_content for d in docs
    )

    summary = llm.invoke(
        "In 2 short lines, what is this document about?\n\n"
        + full_text[:3000]
    ).content

    KB.update(
        store=store,
        name=name,
        summary=summary.strip(),
        pages=len(docs),
        chunks=len(chunks)
    )

    return KB


def clear_knowledge_base():
    KB.update(
```

```
        store=None,
        name=None,
        summary="",
        pages=0,
        chunks=0
    )
```

Code — Part 8: RAG LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class RagState(TypedDict):
    question: str
    history: str
    category: str
    context: str
    sources: list
    answer: str


def rag_classify(state: RagState):

    has_doc = KB["store"] is not None

    doc_info = ""

    if has_doc:
        doc_info = (
            f"The user has uploaded a document named "
            f"'{KB['name']}'. "
            f"It is about: {KB['summary']}\n"
            "Choose 'document' if the question can be answered "
            "from this document or refers to it.\n"
        )

    options = (
        "'document', 'tech' or 'general'"
        if has_doc
        else "'tech' or 'general'"
    )

    res = llm.invoke(
        f"{doc_info}"
        f"Classify the user's latest question as {options}. "
        f"Reply with one word only.\n\n"
        f"Conversation so far:\n"
        f"{state['history'] or 'None'}\n\n"
        f"Latest question: {state['question']}"
    ).content.lower()

    if has_doc and "document" in res:
        return {"category": "document"}

    return {
        "category": "tech"
        if "tech" in res
```

```
        else "general"
    }


def retrieve(state: RagState):

    query = state["question"]

    if state["history"] and len(query.split()) < 6:

        last_user = [
            l for l in state["history"].split("\n")
            if l.startswith("User:")
        ][-1:]

        query = " ".join(last_user) + " " + query

    results = KB["store"].similarity_search_with_score(
        query,
        k=4
    )

    context = "\n\n".join(
        f"[Page {d.metadata.get('page', '?')}] "
        f"{d.page_content}"
        for d, _ in results
    )

    sources = [
        {
            "page": d.metadata.get("page", "?"),
            "score": round(float(s), 3),
            "text": d.page_content[:220].replace("\n", " ")
        }
        for d, s in results
    ]

    return {
        "context": context,
        "sources": sources
    }


def doc_answer(state: RagState):

    ans = llm.invoke(
        "You are a helpful assistant answering questions "
        "about a document.\n"
        "Use ONLY the context below. "
        "If the answer is not in the context, say "
        "'This is not mentioned in the document.' "
        "Mention the page number(s) you used.\n\n"
        f"Context:\n{state['context']}\n\n"
        f"Conversation so far:\n"
        f"{state['history'] or 'None'}\n\n"
```

```
        f"Question: {state['question']}"
    )

    return {
        "answer": ans.content
    }


def rag_tech(state: RagState):

    ans = llm.invoke(
        "You are a technical expert. "
        "Answer clearly and briefly.\n\n"
        f"Conversation so far:\n"
        f"{state['history'] or 'None'}\n\n"
        f"Question: {state['question']}"
    )

    return {
        "answer": ans.content,
        "sources": []
    }


def rag_general(state: RagState):

    ans = llm.invoke(
        "You are a friendly assistant. "
        "Answer briefly.\n\n"
        f"Conversation so far:\n"
        f"{state['history'] or 'None'}\n\n"
        f"Question: {state['question']}"
    )

    return {
        "answer": ans.content,
        "sources": []
    }


g = StateGraph(RagState)

g.add_node("classify", rag_classify)
g.add_node("retrieve", retrieve)
g.add_node("doc_answer", doc_answer)
g.add_node("tech", rag_tech)
g.add_node("general", rag_general)

g.add_edge(START, "classify")

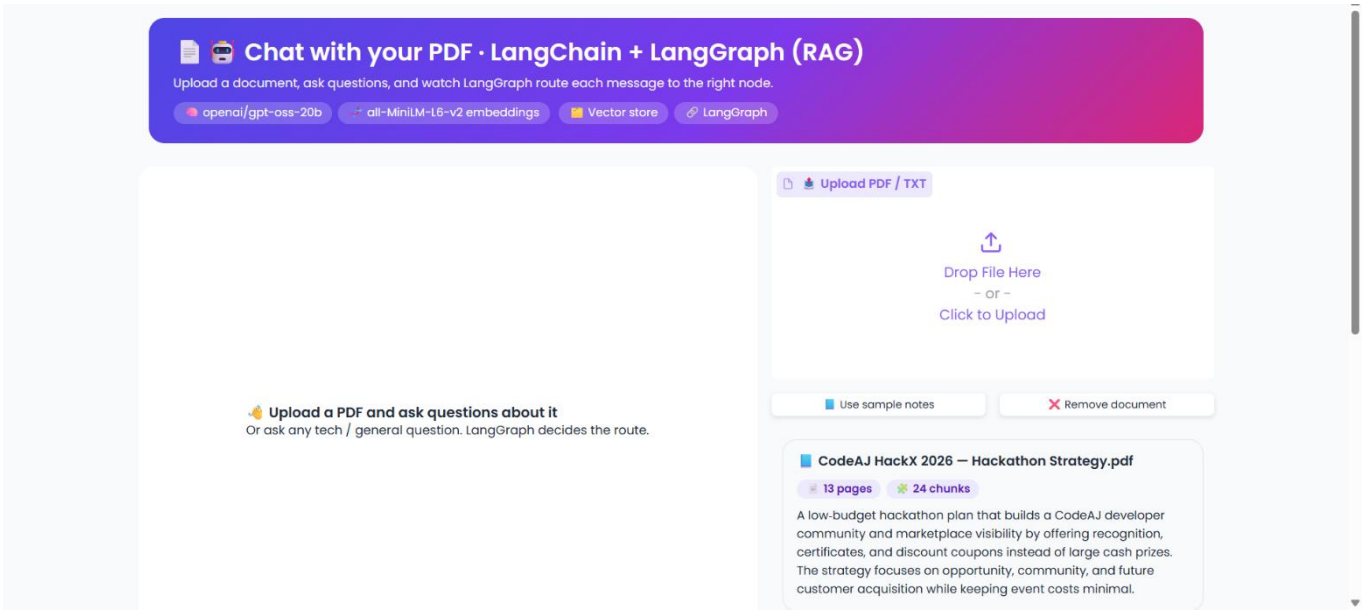
g.add_conditional_edges(
    "classify",
    lambda s: s["category"],
    {
        "document": "retrieve",
```

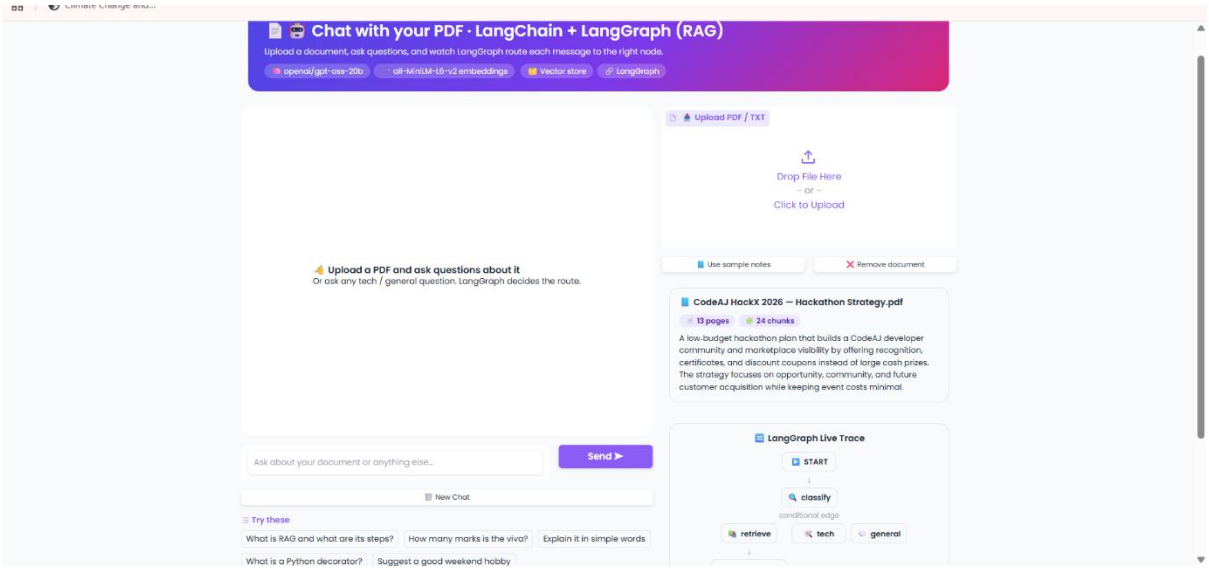
```
        "tech": "tech",
        "general": "general"
    }
)

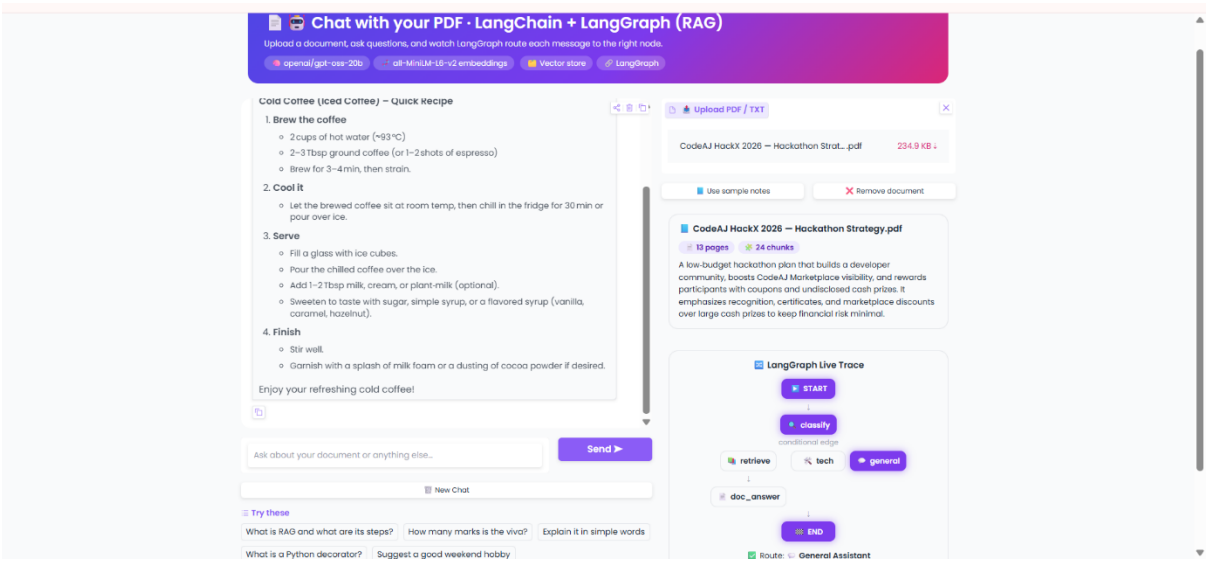
g.add_edge("retrieve", "doc_answer")
g.add_edge("doc_answer", END)
g.add_edge("tech", END)
g.add_edge("general", END)

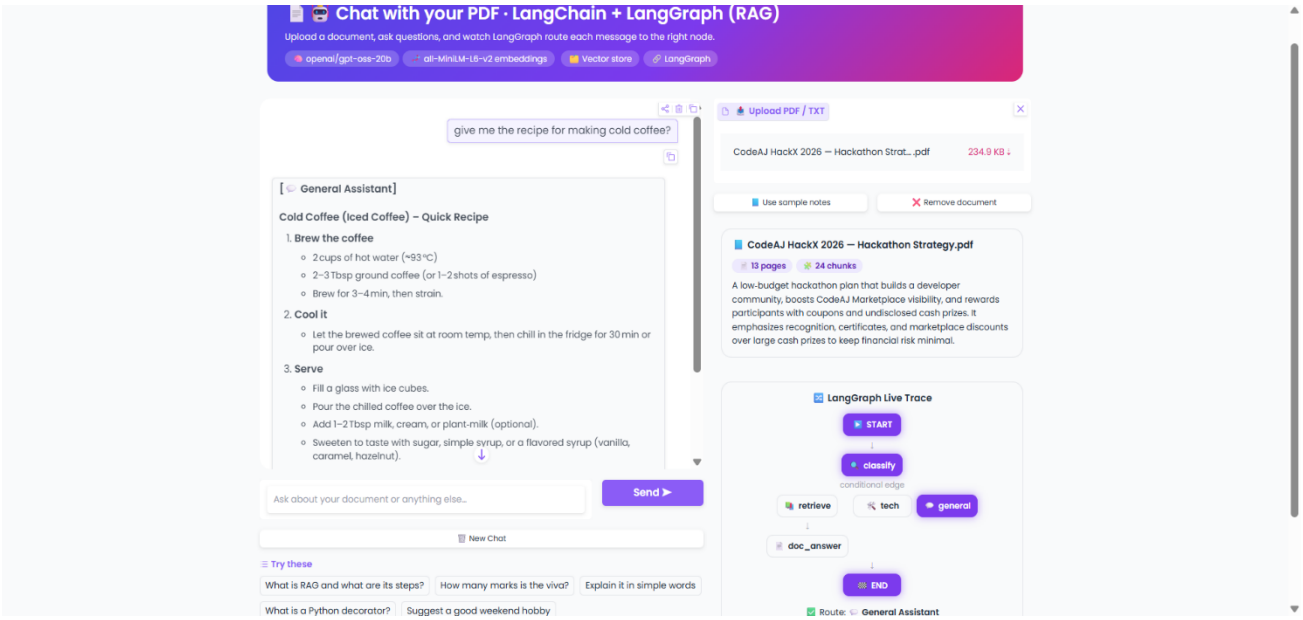
rag_app = g.compile()
```

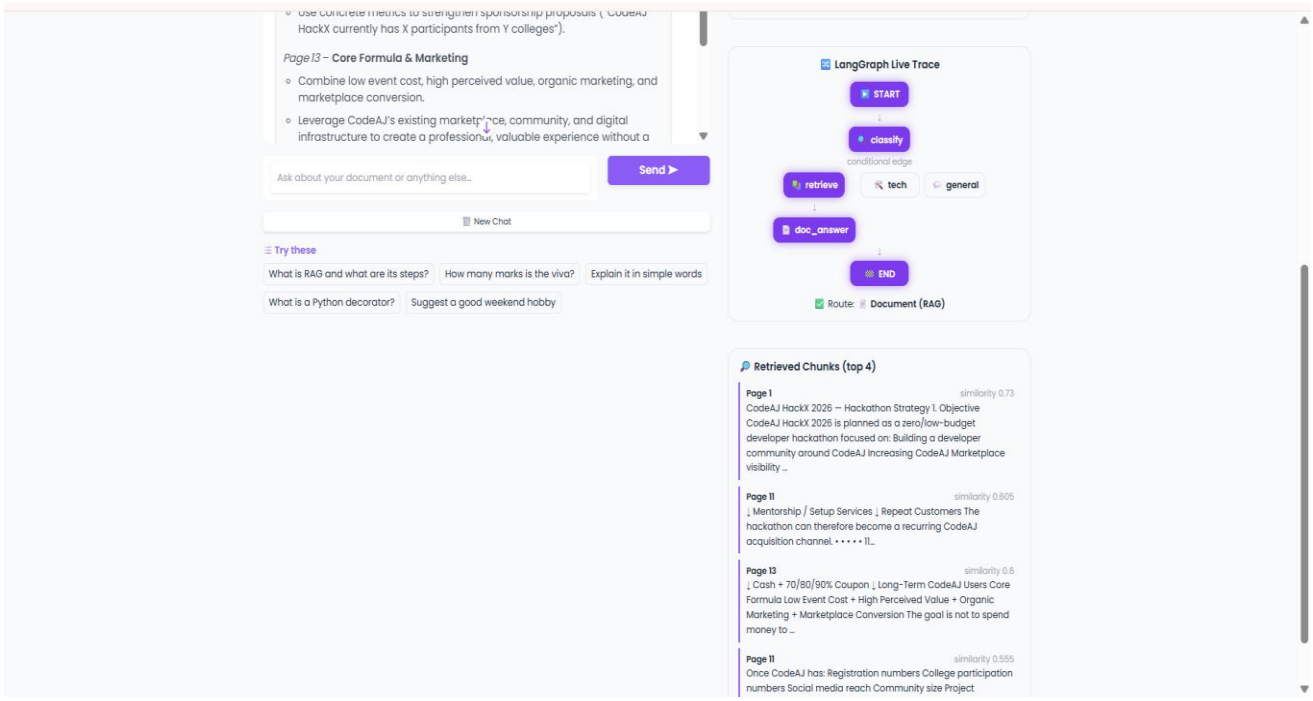
Output






Chat with your PDF · LangChain + LangGraph (RAG)
Upload a document, ask questions, and watch LangGraph route each message to the right node.
openai/gpt-oss-20b
all-MiniLM-L6-v2 embeddings
Vector store
LangGraph
Cold Coffee (Iced Coffee) – Quick Recipe
1. Brew the coffee
2 cups of hot water (~93°C)
2–3 Tbsp ground coffee (or 1–2 shots of espresso)
Brew for 3–4 min, then strain.
2. Cool it
Let the brewed coffee sit at room temp, then chill in the fridge for 30 min or pour over ice.
3. Serve
Fill a glass with ice cubes.
Pour the chilled coffee over the ice.
Add 1–2 Tbsp milk, cream, or plant-milk (optional).
Sweeten to taste with sugar, simple syrup, or a flavored syrup (vanilla, caramel, hazelnut).
4. Finish
Stir well.
Garnish with a splash of milk foam or a dusting of cocoa powder if desired.
Enjoy your refreshing cold coffee!
Upload PDF / TXT
CodeAJ HackX 2026 — Hackathon Strat...pdf
234.9 KB
Use sample notes
Remove document
CodeAJ HackX 2026 — Hackathon Strategy.pdf
13 pages
24 chunks
A low-budget hackathon plan that builds a developer community, boosts CodeAJ Marketplace visibility, and rewards participants with coupons and undisclosed cash prizes. It emphasizes recognition, certificates, and marketplace discounts over large cash prizes to keep financial risk minimal.
LangGraph Live Trace
START
classify
conditional edge
retrieve
tech
general
doc_answer
END
Route: General Assistant
Ask about your document or anything else...
Send
New Chat
Try these
What is RAG and what are its steps?
How many marks is the viva?
Explain it in simple words
What is a Python decorator?
Suggest a good weekend hobby


Chat with your PDF · LangChain + LangGraph (RAG)
Upload a document, ask questions, and watch LangGraph route each message to the right node.
openai/gpt-oss-20b
all-MiniLM-L6-v2 embeddings
Vector store
LangGraph
give me the recipe for making cold coffee?
[General Assistant]
Cold Coffee (Iced Coffee) – Quick Recipe
1. Brew the coffee
2 cups of hot water (~93°C)
2–3 Tbsp ground coffee (or 1–2 shots of espresso)
Brew for 3–4 min, then strain.
2. Cool it
Let the brewed coffee sit at room temp, then chill in the fridge for 30 min or pour over ice.
3. Serve
Fill a glass with ice cubes.
Pour the chilled coffee over the ice.
Add 1–2 Tbsp milk, cream, or plant-milk (optional).
Sweeten to taste with sugar, simple syrup, or a flavored syrup (vanilla, caramel, hazelnut).
Upload PDF / TXT
CodeAJ HackX 2026 — Hackathon Strat...pdf
234.9 KB
Use sample notes
Remove document
CodeAJ HackX 2026 — Hackathon Strategy.pdf
13 pages
24 chunks
A low-budget hackathon plan that builds a developer community, boosts CodeAJ Marketplace visibility, and rewards participants with coupons and undisclosed cash prizes. It emphasizes recognition, certificates, and marketplace discounts over large cash prizes to keep financial risk minimal.
LangGraph Live Trace
START
classify
conditional edge
retrieve
tech
general
doc_answer
END
Route: General Assistant
Ask about your document or anything else...
Send
New Chat
Try these
What is RAG and what are its steps?
How many marks is the viva?
Explain it in simple words
What is a Python decorator?
Suggest a good weekend hobby


HackX currently has X participants from Y colleges").
Page 13 – Core Formula & Marketing
Combine low event cost, high perceived value, organic marketing, and marketplace conversion.
Leverage CodeAJ's existing marketplace, community, and digital infrastructure to create a professional, valuable experience without a
Ask about your document or anything else...
Send ➤
New Chat
Try these
What is RAG and what are its steps?
How many marks is the viva?
Explain it in simple words
What is a Python decorator?
Suggest a good weekend hobby
LangGraph Live Trace
START
classify
conditional edge
retrieve
tech
general
doc_answer
END
Route: Document (RAG)
Retrieved Chunks (top 4)
Page 1
similarity 0.73
CodeAJ HackX 2026 — Hackathon Strategy 1. Objective CodeAJ HackX 2026 is planned as a zero/low-budget developer hackathon focused on: Building a developer community around CodeAJ Increasing CodeAJ Marketplace visibility ...
Page 11
similarity 0.605
↓ Mentorship / Setup Services ↓ Repeat Customers The hackathon can therefore become a recurring CodeAJ acquisition channel. • • • • • 11...
Page 13
similarity 0.6
↓ Cash + 70/80/90% Coupon ↓ Long-Term CodeAJ Users Core Formula Low Event Cost + High Perceived Value + Organic Marketing + Marketplace Conversion The goal is not to spend money to ...
Page 11
similarity 0.555
Once CodeAJ has: Registration numbers College participation numbers Social media reach Community size Project